

AI BlogNest API

AI-PoweredBlogManagementand Content Generation REST API Platform

PHASE 5: PROJECT DEVELOPMENT

Role	Team Member
Team Leader	Praveen Kumar M
Team Member 1	Prabu R
Team Member 2	Muralidharan S
Team Member 3	Yadhava Moorthy Y

Submitted as a phase-wise project submission.

1. Environment and Dependency Setup

The backend application environment was configured using Node.js and key open-source packages managed via npm:

- `express`: Web framework for REST API routing and middleware composition.
- `mongoose`: Object Data Modeling (ODM) library for MongoDB interaction.
- `dotenv`: Zero-dependency module that loads environment variables from a `.env` file.
- `jsonwebtoken`: Implementation of JSON Web Tokens for stateless authorization.
- `bcryptjs`: Password-hashing function for secure credential storage.
- `@google/generative-ai`: Official SDK for integrating Google Gemini AI models.
- `cors` & `morgan`: Cross-Origin Resource Sharing handling and HTTP request logging.

Server Execution Entry Point (`server.js`)

```
const dotenv = require('dotenv');
const app = require('./app');
const connectDB = require('./config/db');

dotenv.config();
connectDB();

const PORT = process.env.PORT || 5000;
app.listen(PORT, () => {
  console.log(`Server running on port ${PORT}`);
});
```

Application Configuration (`app.js`)

```
const express = require('express');
const cors = require('cors');
const morgan = require('morgan');
const authRoutes = require('./routes/authRoutes');
const blogRoutes = require('./routes/blogRoutes');
```



```
const aiRoutes = require('./routes/aiRoutes');
const errorMiddleware = require('./middleware/errorMiddleware');

const app = express();

app.use(cors());
app.use(express.json());
app.use(morgan('dev'));

app.use('/api/auth', authRoutes);
app.use('/api/blogs', blogRoutes);
app.use('/api/ai', aiRoutes);

app.use(errorMiddleware);

app.get('/', (req, res) => {
  res.json({ message: 'AI BlogNest API is running' });
});

module.exports = app;
```

2. Authentication Development

The authentication module features secure user registration and login routines. Passwords are hashed using bcryptjs prior to saving user documents in MongoDB. Upon valid authentication, the server generates a signed JWT payload storing user ID and role details for subsequent authorized requests.

3. AI Features Development

Google Gemini AI models are integrated via a centralized service utility (` geminiService.js `) to handle generative content requests:

- **` POST /api/ai/generate-blog `** : Synthesizes full blog articles given a topic title and target category prompt.
- **` POST /api/ai/summarize `** : Accepts lengthy text content and returns structured executive summaries.

4. Administrator Features

Administrative control routes allow privileged users to manage platform content, enforce comment moderation flags, and adjust user roles dynamically.